

* MICRO BASIC PLUS SOURCE LISTING

*

* MICRO BASIC PLUS

* COPYRIGHT (C) 1976 BY

*

* TECHNICAL SYSTEMS CONSULTANTS

* BOX 2574

* W. LAFAYETTE INDIANA 47906

*

*

* EQUATES

STACK EQU \$A07F

PIAADR EQU \$8004

PFILBG EQU \$A002

PFILEN EQU \$A004

EXTERN EQU \$1F00

MONITR EQU \$E0E3

MONPC EQU \$A048

STKBOT EQU \$A000

* TEMPORARY STORAGE

RNDM RMB 4

BUFPNT RMB 2

FORSTK RMB 2

DIMPNT RMB 2

XTEMP3 RMB 2

DATAST RMB 2

DATAPT RMB 2

TRYVAL RMB 2

CRFLAG RMB 1

QMFLAG RMB 1

ROWVAR RMB 1

ROWCON RMB 1

COLCON RMB 1

TABFLG RMB 1

DIMFLG RMB 1

RUNFLG RMB 1

DATAFL RMB 1

SUBCNT RMB 1

LETFLG RMB 1

FLDCNT RMB 1

NXPNTR RMB 2

XTEMP RMB 2

XSAVE RMB 2

XSAVE2 RMB 2

NUMCNT RMB 1

NEGFLG RMB 1

NOEXFL RMB 1
EXTRA RMB 2
COUNT RMB 1
STKCNT RMB 1
AUXCNT RMB 1
SIGN RMB 1
AXSIGN RMB 1
OVFLBF RMB 1
XTEMP2 RMB 2
XTEMP4 RMB 2
XTEMP5 RMB 2
CPX1 RMB 2
CPX2 RMB 2
STKEND RMB 3
CHRCNT RMB 1
OPSTAK RMB 32
AC RMB 3
NUMBER RMB 3
AX RMB 3
BUFFER RMB 72

* LABEL TABLE

LBLTBL RMB 78
STKTOP RMB 2

* CONSTANTS

BACKSP EQU \$8
DELCOD EQU \$18
PRMPTC EQU \$21

ORG \$0100

* MAIN PROGRAM

START JMP MICBAS JMP TO BEGIN
RESTRT JMP FILBUF

* EXTERNAL I-O ROUTINES

OUTEEE JMP \$E1D1
INCH JSR \$E1AC
BREAK JMP INTBRK
MEMEND FDB \$1EFF

* KEYWORD AND JUMP TABLE

KEYTBL FCC ;PRI;

FDB PRINT

FCC ;INP;
FDB INPUT

FCC ;IF ;
FDB IF

FCC ;LET;
LETADR FDB LET

FCC ;FOR;
FDB FOR

FCC ;NEX;
FDB NEXT

FCC ;GOT;
FDB GOTO

FCC ;GOS;
FDB GOSUB

FCC ;ON ;
FDB ONGOTO

FCC ;RET;
FDB RETURN

FCC ;REA;
FDB READ

FCC ;DAT;
FDB DATA

FCC ;RES;
FDB RESTOR

FCC ;DIM;
FDB DIM

FCC ;EXT;
FDB EXTRNL

FCC ;MON;
FDB MONITR

FCC ;END;
FDB FILBUF

FCC ;REM;
FDB RUNEXC

FCC ;RUN;
FDB RUN

FCC ;LIS;
FDB LIST

FCC ;SCR;
FDB MICBAS
FCB 0

FCTTBL FCC ;RND;
FDB EVAL88

FCC ;ABS;
FDB EVAL85

FCC ;SGN;
FDB EVAL86
FCB 0

* INITIALIZATION

CLRBEG LDX #START
STX XTEMP3 SAVE X
CLRBG2 LDX #DATAST SET START
BRA CLEAR GO CLEAR

CLREND LDX MEMEND SET END
STX XTEMP3 SAVE
LDX ENDSTR
CLEAR CLR A CLEAR ACC.
CLEAR2 STA A 0,X CLEAR BYTE
INX BUMP THE POINTER
CPX XTEMP3 DONE?
BNE CLEAR2
RTS RETURN

MICBAS BSR CLRBEG GO CLEAR
LDX #STORSP
STX ENDSTR SET END STORAGE:
BSR CLREND GO CLEAR

* GET LINE INTO INPUT BUFFER

FILBUF LDX #RESTRT

```

STX MONPC SET UP RETURN POINTER
LDS #STACK
LDX #BUFFER
STX XTEMP3 SAVE BOUND
BSR CLRBG2
LDX #ENDSTR SET PUHCH LIMITS
STX PFILBG
LDX 0,X SET END
STX PFILEN
STX DIMPNT
LDX #BUFFER POINT TO BUFFER
JSR PCRLF OUT A CR & LF
LDA A #PRMPTC
JSR OUTCH OUTPUT PROMPT
FILBU2 JSR INCHAR GET A CHARACTER
BEQ FILBUF
STA A 0,X SAVE CHAR.
CMP A #$0D IS IT A C.R. ?
BEQ FILBU6
INX BUMP THE POINTER
CPX #BUFFER+72
BNE FILBU2 END OF BUFFER?
BRA FILBUF
FILBU6 LDX #BUFFER RESET POINTER
JSR BCDC01 LINE NO. CONV.
STX XTEMP2 SAVE POINTER
JSR FNDKEY CHECK KEY WORD
TST A
BNE FILBU8 IF NONZERO THEN OK
LDX BUFPNT POINT TO BUFFER
LDA A 0,X GET CHARACTER
CMP A #$D IS IT A C.R.?
BNE FILBU7
LDA B NOEXFL DIR. EXECUTION?
BEQ FILBUF
STA A CRFLAG SET FLAG
BRA FILBU8 IT IS OK
FILBU7 JSR TSTLET LET?
BEQ FILBU8
FILB75 LDA A #$10
JMP MISTAK REPORT ERROR #0
FILBU8 LDA A CHRCNT GET CHAR. COUNT
SUB A NUMCNT SUB LINE # DIGITS
STA A CHRCNT SAVE
LDA B NOEXFL DIRECT EXECUTE ?
BNE STUFLN IF NOT GO PUT LINE
JSR PCRLF OUTPUT C.R. L.F.
JMP RUNEX4 GO TO ROUTINE

```

* PUT LINE IN PROGRAM STORAGE

```
STUFLN LDX MEMEND
STX CPX1
LDX XTEMP2 SET POINTER
STX BUFPNT SAVE POINTER
JSR FNDLIN GO FIND LINE IN STORE
STX XSAVE SAVE POINTER
TST B DID WE FIND IT?
BNE INSERT IF NOT GO INSERT
```

* REPLACE EXISTING LINE WITH NEW ONE

```
REPLAC INC B INC THE COUNTER
LDA A 0,X GET A CHARACTER
INX BUMP THE POINTER
CMP A #$D IS IT A C.R.,?
BNE REPLAC
REPLA4 STA B OFFSET2+1 SETUP OFFSET
LDA A #$FF GET COUNT
NEG B 2'S COMP. IT
BSR ADJEND GO FIX END PNTR
LDX XSAVE RESTORE THE POINTER
REPLA5 CPX ENDSTR END OF STORAGE?
BEQ REPLA6
OFFSET2 LDA A 0,X
STA A 0,X MOVE A CHARACTER
INX BUMP THE POINTER
BRA REPLA5 REPEAT
REPLA6 LDX XSAVE RESTORE THE POINTER
```

* INSERT A LINE INTO PROGRAM STORAGE

```
INSERT LDA A CRFLAG LONE C.R. ?
BNE INSERT6
LDX ENDSTR
LDA B CHRCNT GET CHAR. COUNT
ADD B #2 BIAS FOR LINE NUM.
STA B OFFSET+1 SETUP OFFSET
BSR ADJEND FIX END PNTR
INSERT2 CPX XSAVE DONE?
BEQ INSERT3
DEX DEC THE POINTER
LDA A 0,X GET A CHAR,
OFFSET STA A 0,X
BRA INSERT2 MOVE IT
INSERT3 DEX
JSR PUTLB2 PUT LAB
```

```

    INX BUMP THE POINTER
    INX
INSERT4 STX XSAVE SAVE POINTER
    LDX BUFPNT
    LDA A 0,X GET CHAR*
    INX BUMP THE POINTER
    STX BUFPNT SAVE
    LDX XSAVE RESTOR PNTR
    INX
    STA A 0,X SAVE IT
    CMP A #$D IS IT A C.R.?
    BNE INSERT4
INSERT6 JMP FILBUF 60 TO MAIN LOOP

```

* ADJUST THE END OF PROGRAM POINTER

```

ADJEND ADD B ENDSTR+1
    ADC A ENDSTR ADD IN VALUE
    STA B CPX2+1
    STA A CPX2 SET END POINTER
    JSR CMPX1
    BCC ADJEN2
    STA B ENDSTR+1
    STA A ENDSTR SAVE NEW POINTER
    RTS RETURN
ADJEN2 LDA A #$90 SET ERROR
    JMP MISTAK

```

* TRY TO FIND LINE

```

FNDLIN LDA A NUMBER+2
    LDA B NUMBER+1
FINDLN LDX #STORSP SETUP POINTER
FINDL1 CPX ENDSTR END OF STORAGE?
    BNE FINDL4
FINDL2 INC B
    RTS RETURN
FINDL4 CMP B 0,X CHECK M.S. DIGITS
    BHI FINDL6
    BNE FINDL2
    CMP A 1,X CHECK L.S, DIGITS
    BHI FINDL6
    BNE FINDL2
    CLR B CEAR FLAG
    RTS RETURN
FINDL6 BSR FND CRT GO FIND C.R,
    INX BUMP THE POINTER
    BRA FINDL1 REPEAT

```

* FIND A C,R, IN STORAGE

```
FNDCRT PSH A SAVE A
      LDA A #$D
FNDVAL INX BUMP THE POINTER
      CMP A 0,X TEST FOR C.R.
      BNE FNDVAL
      PUL A RESTORE A
      RTS RETURN
```

* INPUT

```
INCHAR JSR INCH GET THE CHAR.
      CMP A #BACKSP IS IT A BACKSPACE?
      BNE INCHR2
      CPX #BUFFER BEGINNING OF BUF ?
      BEQ INCHR4
      DEX BACKUP ONE POS.
      DEC CHRCNT DEC CHAR. COUNT
      BRA INCHAR
INCHR2 CMP A #DELCOD DELETE LINE ?
      BEQ INCHR4
      INC CHRCNT
INCHR4 RTS RETURN
```

* PRINT CARRIAGE RETURN & LINEFEED

```
PCRLF STX XSAVE SAVE X REG
      LDX #CRLFST POINT TO STRING
PDATA1 LDA A 0,X GET CHAR
      CMP A #4 IS IT 4?
      BEQ PCRLF2
      JSR OUTCH OUTPUT CHAR
      INX BUMP THE POINTER
      BRA PDATA1 REPEAT
PCRLF2 LDX XSAVE RESTORE X REG
      CLR FLDCNT ZERO FIELD COUNT
      RTS RETURN
```

CRLFST FCB \$D,\$A,0,0,0,0,4

* TEST FOR STATEMENT TERMINATOR

```
TSTTRM CMP A #$D C,R,?
      BEQ TSTTR2
      CMP A #' : COLON?
TSTTR2 RTS RETURN
```

* CLEAR NUMBER THROUGH NUMBER+2


```
UPSCLR JSR STAKUP
CLRNUM CLR A
STA A NUMBER
STA A NUMBER+1
STA A NUMBER+2
RTS
```

* CONVERT NUMBER TO PACKED BCD

```
BCDCON BSR CLRNUM CLEAR NUMBER
STA A NOEXFL
STA A NEGFLG
STA A NUMCNT
JSR SKIPSP SKIP SPACES
CMP A #' + IS IT A + ?
BEQ BCDC01
CMP A #' - IS IT A - ?
BNE BCDC01
COM NEGFLG SET FLAG
BCDC01 INX
BCDC01 JSR CLASS GET A DIGIT
CMP B #3 IS IT A NUMBER?
BEQ BCDC02
LDA A NEGFLG
JMP FIXSIN GO FIX UP THE SIGN
BCDC02 INX BUMP THE POINTER
STA A NOEXFL SET NO EXEC FLU
AND A #$0F MASK OFF ASCII
LDA B #4 SET COUNTER
BCDC04 ASL NUMBER+2
ROL NUMBER+1
ROL NUMBER SHIFT PREV. OVER
DEC B DEC THE COUNTER
BNE BCDC04
ADD A NUMBER+2
STA A NUMBER+2 SAVE NEW VALUE
INC NUMCNT INC NUMBER CNTR
BRA BCDC01
```

* FIND NEXT BLOCK

```
NXTBLK LDX BUFPNT RESTORE POINTER
NXTBL4 LDA A 0,X GET A CHAR.
CMP A #' IS IT A SPACE?
BEQ SKIPSP
INX BUMP THE POINTER
BRA NXTBL4 REPEAT
```

* CONVERT AND SKIP

```
CONSKP BSR BCDCON  
DEX
```

* SKIP ALL SPACES

```
SKPSP0 INX  
SKIPSP LDA A 0,X GET CHR FROM BUF  
CMP A #$20 IS IT A SPACE?  
BEQ SKPSP0  
SKIPS4 RTS RETURN
```

* FIND NEXT BLOCK NOT EXPECTING A SPACE

```
NXTSPC LDX BUFPNT SET POINTER  
NXTSP4 JSR CLASS GO CLASSIFY  
CMP B #2 IS IT A LETTER?  
BNE SKIPSP  
INX BUMP THE POINTER  
BRA NXTSP4
```

* FIND KEY WORD IF POSSIBLE

```
FNDKEY JSR SKIPSP SKIP SPACES  
STX BUFPNT SAVE THE POINTER  
STX XSAVE  
LDX #KEYTBL POINT TO KEY WORDS  
FNDKE2 LDA B #5  
FNDKE4 CMP A 0,X TEST THE CHARACTER  
BNE FNDKE6  
STX XTEMP3 SAVE POINTER  
LDX XSAVE  
INX BUMP POINTER  
LDA A 0,X GET CHAR.  
STX XSAVE  
LDX XTEMP3 REST. PNTR.  
INX  
DEC B  
CMP B #2  
BNE FNDKE4 IF NOT DONE REPEAT  
FNDKE5 RTS RETURN  
FNDKE6 INX BUMP THE COUNTER  
DEC B  
BNE FNDKE6  
LDA A 0,X GET CHARACTER  
BEQ FNDKE5 IF ZERO, END OF LIST  
STX XTEMP3 SAVE POINTER  
LDX BUFPNT
```

```
STX XSAVE
LDA A 0,X GET NEW CHAR.
LDX XTEMP3 RESTORE POINTER
BRA FNDKE2 REPEAT
```

* OUTPUT A NUMBER FROM PACKED BCD BYTES

```
OUTBCD LDX #NUMBER SET POINTER
OUTBCI LDA B #2 SET COUNTER
CLC
LDA A 0,X GET A WORD
BPL OUTBC4 IF NOT NEG JMP AHEAD
LDA A #' -
JSR OUTCH OUTPUT A
INC FLDCNT
BRA OUTBC4
OUTBC2 LDA A 0,X GET DIGITS
BIT A #$F0 MASK
BCS OUTBC3
BEQ OUTBC4 JMP IF ZEROES
OUTBC3 JSR OUTHL OUTPUT A DIGIT
INC FLDCNT
SEC
OUTBC4 LDA A 0,X GET A DIGIT
BIT B #$FF LAST DIGIT?
BEQ OUTBC6
BIT A #$0F MASK
BCS OUTBC6
BEQ OUTBC8 JMP IF ZEROES
OUTBC6 JSR OUTHR OUTPUT A DIGIT
INC FLDCNT
SEC
OUTBC8 INX BUMP THE POINTER
DEC B DEC THE COUNTER
BPL OUTBC2 REPEAT IF NOT DONE
RTS RETURN
```

* LIST USERS PROGRAM

```
LIST JSR NXTSPC FIND NEXT
CMP A #$D
BEQ LIST3
JSR BCDCON GET LINE NUM
STX BUFPNT SAVE POINTER
JSR FNDLIN FIND LINE
STX XSAVE SAVE IT
JSR NXTSPC
CMP A #$D C.R.?
```

```

BNE LIST1
INC SUBCNT SET TO 1
BRA LIST2
LIST1 INX BUMP THE POINTER
JSR SKIPSP
JSR BCDCON GET COUNT
LDA A NUMBER+2
STA A SUBCNT SAVE IT
LIST2 LDX XSAVE POINT TO LINE
BRA LIST4
LIST3 LDX #STORSP SET POINTER
LIST4 CPX ENDSTR END OF STORAGE?
BEQ LIST8
JSR PCRLF OUTPUT A
LDA B #1 SETUP COUNTER
CLC
BSR OUTBC2 OUT LINE NUMBER
LIST5 LDA A 0,X GET A CHARACTER
CMP A #$D IS IT A C.R.?
BEQ LIST6
BSR OUTCH OUTPUT CHARACTER
INX BUMP THE POINTER
BRA LIST5 REPEAT
LIST6 INX BUMP THE POINTER
LDA A SUBCNT GET COUNT
BEQ LIST4
ADD A #$99 DEC THE COUNT
DAA
BEQ LIST8
STA A SUBCNT SAVE
BRA LIST4
LIST8 JMP FILBUF

```

```

OUTH LSR A
LSR A
LSR A
LSR A MOVE TO BOTTOM
OUTHR AND A #$0F MASK
ADD A #$30 BIAS
OUTCH JSR BREAK CHECK FOR BREAK
JMP OUTEEE GO PRINT

```

* INTERNAL BREAK ROUTINE

```

INTBRK PSH A
LDA A PIAADR CHECK
BPL BREAK2
PUL A GET CHAR
RTS RETURN

```

```
BREAK2 LDA A PIAADR
      BPL BREAK2
      LDA A #$99 SET ERROR
```

* OUTPUT ERROR MESSAGE

```
MISTAK PSH A SAVE A
      JSR PCRLF OUTPUT A CR & LF
MISTA1 LDX #ERRSTR POINT TO ERROR STRING
      JSR PDATA1 OUTPUT IT
      PUL A RESTORE A
      PSH A SAVE A
      JSR OUTHL OUTPUT DIGIT
MISTA2 PUL A RESTORE A
      JSR OUTHR OUT 1'S DIGIT
      LDA B RUNFLG RUNNING?
      BNE RUNER1
MISTA4 JMP FILBUF
RUNER1 LDX #ERSTR2 POINT TO STRING
      JSR PDATA1 OUTPUT IT
      LDX BUFPNT SET POINTER
RUNER2 DEX DEC THE POINTER
      CPX #STORSP BEGINNING?
      BEQ RUNER4
      LDA A 0,X GET CHAR
      CMP A #$D C.R.?
      BNE RUNER2
      INX BUMP THE POINTER
RUNER4 LDA B #1
      CLC
      JSR OUTBC2 OUT LINE NUM.
      BRA MISTA4
ERRSTR FCB 7
      FCC ;ERROR #;
      FCB 4
```

```
ERSTR2 FCC ; AT ;
      FCB 4
```

* PRINT ROUTINE

```
PRINT JSR NXTSPC FIND NEXT BLOCK
PRINT0 JSR TSTTRM
      BNE FIELD1
      JMP PRINT8
FIELD1 CLR CRFLAG
      CMP A #', IS IT A ", "
      BNE PRINT2
      LDA B FLDCNT GET COUNT
```

```

FIELD2 LDA A #' SPACE
      JSR OUTCH OUTPUT A SPACE
      INC B
      BIT B #7 END OF FIELD?
      BNE FIELD2
      CMP B #$47 END OF LINE?
      BHI FIELD3
      STA B FLDCNT SAVE FIELD INFO
      BRA PRINT1
FIELD3 JSR PCRLF OUT A C.R. & L.F.
PRINT1 INC CRFLAG SET FLAG
      INX BUMP THE POINTER
      JSR SKIPSP
      BRA PRINT0
PRINT2 CMP A #'; IS IT A ";"
      BEQ PRINT1
      CMP A #' " IS IT A QUOTE?
      BNE PRINT4
      INX BUMP THE POINTER
      BSR PSTRNG OUTPUT STRING
      BRA PRINT6
PRINT4 CLR TABFLG CLEAR FLAG
      CMP A #'T IS IT A T?
      BNE PRIN45
      STA A TABFLG SET FLAG
      LDA A #'A
      BRA PRIN47
PRIN45 CMP A #'S IS IT A S?
      BNE PRIN55
      LDA A #'P
PRIN47 CMP A 1,X
      BNE PRIN55
      JSR NXTSP4 FIND NEXT
      JSR EXPR EVALUATE
      JSR BINCON CONVERT
      LDA B NUMBER+2
      BEQ PRINT6
      LDA A TABFLG CHECK FLAG
      BEQ PRINT5
      DEC B
      CMP B FLDCNT CHECK COUNT
      BLS PRINT6
      BRA PRIN51
PRINT5 ADD B FLDCNT
PRIN51 LDA A #' SPACE
      JSR OUTCH OUTPUT SPACE
      INC FLDCNT BUMP COUNTER
      CMP B FLDCNT
      BNE PRIN51 REPEAT

```

```

PRIN52 BRA PRINT6
PRIN55 JSR EXPR EVAL EXPRESSION
      STX XSAVE SAVE POINTER
      JSR OUTBCD OUTPUT VALUE
      LDX XSAVE RESTORE
PRINT6 JSR SKYCLS
      DEC B
      BNE PRINT7 CHECK FOR ERROR
      JMP PRINT0
PRINT7 LDA A #$31
      JMP MISTAK
PRINT8 TST CRFLAG C.R. ?
      BNE PRINT9
      JSR PCRLF OUTPUT C.R. L.F
PRINT9 JMP RUNEXC

```

* PRINT STRING ROUTINE

```

PSTRNG LDA A 0,X GET A CHAR.
      CMP A #' " IS I T A QUOTE?
      BEQ PSTRN4
      JSR TSTTRM IS IT A C.R.?
      BEQ PSTRN8
      JSR OUTCH OUTPUT CHARACTER
      INC FLDCNT BUMP FIELD CNT
      INX BUMP THE POINTER
      BRA PSTRNG REPEAT
PSTRN4 INX
      JMP SKIPSP
PSTRN8 LDA A #$32
      JMP MISTAK REPORT ERROR

```

* FIND LABEL ROUTINE

```

FNDVAR STX BUFPT SAVE POINTER
      JSR CLASS1 GO CLASSIFY CHAR.
      CMP B #2 CHECK FOR LETTER
      BNE FNDL25 ERROR
      CLR XTEMP
      TAB SAVE LABEL
      ASL A MULT IT BY 2
      ABA ADD IT
      SUB A #$13
      STA A XTEMP+1
      LDX XTEMP POINT TO IT
      RTS RETURN

```

* FIND DIMENSIONED VARIABLE

```

FNDLB0 LDA A 0,X
FNDLBL INX ADVANCE POINTER
CLR DIMFLG
BSR FNDVAR GO FIND VAR.
CLR B
LDA A 0,X GET CHAR.
CMP A #$0A CHECK FOR 1 DIM
BEQ FNDLB2
CMP A #$0B CHECK IF 2 DIM
BEQ FNDLB1
RTS
FNDLB1 INC B SET FLAG-2 DIM
FNDLB2 LDA A 1,X SET POINTER
PSH A
LDA A 2,X
PSH A
PSH B SAVE B
JSR NXTSPC FIND NEXT
PUL B
CMP A #'( IS IT A PAREN?
FNDL25 BNE FNDLB9
TST B
BEQ FNDLB3
INX
JSR EXPRO GO EVALUATE
LDA A NUMBER+2 GET RESULT
PSH A SAVE IT
JSR STAKDN RESTORE
JSR NXTSPC FIND NEXT
CMP A #', IS IT A COMMA?
BNE FNDLB9
BRA FNDLB4
FNDLB3 CLR A
PSH A SET ROWV
FNDLB4 INC A
STA A DIMFLG SET FLAG
INX
JSR EXPRO
INX
STX BUFPNT SAVE POINTER
PUL A
STA A ROWVAR SAVE
PUL A
STA A XTEMP+1 SAVE
PUL A
STA A XTEMP SAVE
LDX XTEMP SET POINTER
LDA A 0,X GET CHAR
STA A COLCON SAVE IT

```



```

INX BUMP THE POINTER
INX
STX XTEMP
JSR UPSCLR
LDA A ROWVAR GET VAR.
LDX XTEMP
DEX DEC POINTER
CMP A 0,X CHECK
BHI FNDLB9
STA A NUMBER+2
JSR UPSCLR PUSH STACK
LDA A COLCON GET CONST,
CMP A AC-1 CHECK
BEQ FNDL45
BLS FNDLB9 ERROR!
FNDL45 ADD A #1
DAA BIAS IT
STA A NUMBER+2
JSR MULT GO MULTIPLY
JSR ADD GO ADD
FNDLB5 JSR TIMTHR

```

* ROUTINE TO ADD VALUE TO X-REG.

```

ADDX LDA A XTEMP GET M.S.BYTE
LDA B XTEMP+1
ADD B NUMBER+2
ADC A NUMBER+1
STA A XTEMP SAVE SUM
STA B XTEMP+1
JSR STAKDN
LDX XTEMP SET POINTER
CLR DIMFLG RESTORE FLAG
RTS RETURN

```

```

FNDLB9 LDA A #$14 SET ERROR
JMP MISTAK GO REPORT

```

* ROUTINE TO MULTIPLY BY 3

```

TIMTHR JSR UPSCLR
LDA A #$3 SET MULTIPLIER
STA A NUMBER+2
JSR MULT GO MULTIPLY

```

* BCD TO BINARY CONVERT.

```

BINCON LDA A NUMBER+2 GET LS BYTE
PSH A SAVE

```

```

LDA A NUMBER+1
PSH A SAVE:
CLR B
STA B NUMBER+1
STA B NUMBER+2 INITIALIZE
LDA A NUMBER
BSR ADSHF1 ADD AND SHIFT
PUL A
PSH A
BSR ADSHF0 GO ADD IN AND SHIFT
PUL A GET MS BYTE AGAIN
BSR ADSHF1 GO ADD IN AND SHIFT
PUL A GET LS BYTE
PSH A
BSR ADSHF0
PUL A
BRA ADDIN GO ADD IN ONES
ADSHF0 LSR A
LSR A
LSR A
LSR A MOVE TO LS HALF
ADSHF1 BSR ADDIN GO ADD IN
LDA B NUMBER+1
ASL A
ROL B MULT BY 2
PSH B
PSH A SAVE
ASL A
ROL B
ASL A
ROL B MULT BY 4, *=8
STA A NUMBER+2
PUL A
STA B NUMBER+1
BSR ADDIN1 GO ADD IN
PUL A
ADD A NUMBER+1
STA A NUMBER+1 MULTIPLY BY TEN
RTS
ADDIN AND A #$0F MASK
ADDIN1 ADD A NUMBER+2
STA A NUMBER+2
BCC ADDIN2 CHECK FOR CARRY
INC NUMBER+1
ADDIN2 RTS

```

* PUT LABEL ROUTINE

```

PUTLBL LDA A NUMBER

```

```
STA A 0,X PUT M.S. BYTE
PUTLB2 LDA A NUMBER +1
STA A 1,X PUT NEXT
LDA A NUMBER+2
STA A 2,X PUT L.S. BYTE
RTS RETURN
```

* DIMENSION

```
DIM LDX FORSTK SET BOUNDS
STX CPX1
JSR NXTSPC
DIMN JSR SKIPSP CLASSIFY
JSR FNDVAR
STX XTEMP3 SAVE IT
JSR NXTSPC GET TO NEXT
CMP A #'( IS IT A PARENT
BNE DIM9
DIM01 INX BUMP THE POINTER
JSR CONSKP CONVERT DIM
CMP A #') IS IT A PAREN
BNE DIM1
CLR A
CLR B
PSH A SAVE IT
BRA DIM2
DIM1 CMP A #', COMMA?
BNE DIM9 ERROR!
LDA A NUMBER+2
BEQ DIM9
PSH A SAVE
INX BUMP THE POINTER
JSR CONSKP CONVERT
LDA B #1
CMP A #') PAREN?
BEQ DIM2
DIM9 LDA A #$40 SET ERROR
JMP MISTAK REPORT
DIM2 LDA A NUMBER+2
BEQ DIM9
PSH A SAVE
STX BUFPNT SAVE POINTER
LDX XTEMP3 SET X
LDA A #$0A
ABA SET MARKER
STA A 0,X SAVE IT
LDA A DIMPNT GET POINTER
STA A 1,X SAVE IT
LDA A DIMPNT+1
```

```

STA A 2,X
LDX DIMPNT SET POINTER
PUL A
STA A 0,X SAVE 1ST DIM
INX BUMP THE POINTER
PUL B
STA B 0,X SAVE 2ND DIM
INX
STX XTEMP SAVE POINTER
ADD A #1
DAA BIAS
PSH A
TBA
ADD A #1 BIAS
DAA ADJUST
TAB SAVE
JSR CLRNUM CLEAR STORAGE
STA B NUMBER+2
JSR UPSCLR GO CLEAR
PUL A
STA A NUMBER+2
JSR MULT MULTIPLY
JSR FNDLB5 GO FIX X
JSR CMPX TEST BOUNDS
BLS DIM5
JMP ADJEN2
DIM5 STX DIMPNT SAVE RESULT
LDX BUFPNT RESTORE F'NTR
INX
JSR SKIPSP SKIP SPACES
JSR TSTTRM
BEQ RUNEXC
INX BUMP THE POINTER
JMP DIMN

```

* EXTERNAL ROUTINE JUMP

EXTRNL JSR EXTERN GO TO IT

* RUN EXECUTIVE

```

RUNEXC CLR A
STA A CRFLAG
STA A LETFLG
STA A DIMFLG
STA A STKCNT
LDA A RUNFLG RUN MODE?
BNE RUNEX0
RUNEXA JMP FILBUF

```

```

RUNEX0 LDX BUFPNT SET POINTER
RUNE05 LDA A #D
    LDA B #':': SETUP TERMINATORS
RUNEX1 CMP A 0,X C.R. ?
    BEQ RUNEX2
    CMP B 0,X IS IT A ':' ?
    BEQ RUNE27
    INX BUMP THE POINTER
    BRA RUNEX1 REPEAT
RUNEX2 INX
RUNE22 CPX ENDSTR END OF STORAGE?
    BEQ RUNEXA
RUNE25 INX BUMP THE POINTER
RUNE27 INX
    JSR BREAK GO CHECK BREAK
RUNEX3 JSR FNDKEY FIND KEY WORD
    TST A
    BNE RUNEX4
    LDX BUFPNT SET POINTER
    BSR TSTLET
    BEQ RUNEX4
    LDA A #10
RUNE35 JMP MISTAK
RUNEX4 LDX 0,X
    JMP 0,X GO TO ROUTINE

```

* TEST FOR IMPLIED LET

```

TSTLET JSR CLASS CHECK CHAR.
    CMP B #2 LETTER?
    BNE TSTLE2
    INX BUMP THE POINTER
    JSR SKIPSP SKIP SPACES
    CMP A #'= EQUALS?
    BEQ TSTLE1
    CMP A #'( LEFT PARENT
    BNE TSTLE2
TSTLE1 LDX #LETADR SET POINTER
    STA A LETFLG SET FLAG
    CLR B
TSTLE2 RTS

```

* RUN ROUTINE

```

RUN JSR CLRBEG
    JSR CLREND
    LDX MEMEND
    STX FORSTK
    LDX #STORSP SET POINTER

```

```
INC RUNFLG
BRA RUNE22
```

* LET ROUTINE

```
LET LDX BUFPNT
LDA A LETFLG TEST FLAG
BNE LET2
JSR NXTBLK FIND NEXT
LET2 JSR EXPEQU
JMP RUNEXC
```

* GOTO ROUTINE

```
GOTO JSR NXTSPC FIND BLOCK
GOTO1 JSR EXPR GO EVALUATE
GOTO2 JSR FNDLIN GO FIND LINE
GOTO3 TST B FIND?
BEQ GOTO5
LDA A #$16 SET ERROR
GOTO4 JMP MISTAK REPORT
GOTO5 INC B
STA B RUNFLG SET RUN FLAG
JMP RUNE22
```

* INPUT ROUTINE

```
INPUT JSR NXTSPC FIND NEXT
INPUT0 CLR QMFLAG CLEAR FLAG
INPUT1 JSR SKIPSP SKIP SPACES
CMP A #' " IS IT A QUOTE?
BNE INPUT2
INX BUMP THE POINTER
JSR PSTRNG OUTPUT STRING
BRA INPUT6
INPUT2 JSR FNDLBL FIND LABEL
STX XTEMP4 SAVE POINTER
INPUT3 LDX #BUFFER SET POINTER
LDA A QMFLAG TEST FLAG
BNE INPUT4
LDA A #' ?
STA A QMFLAG SET FLAG
JSR OUTCH OUT A ?
INPUT4 JSR INCH GET A DIGIT
CMP A #DELCOD DELETE?
BNE INPU45
CLR QMFLAG
BRA INPUT3
INPU45 STA A 0,X SAVE IT
```

```

    INX
    CMP A #' , 1S IT COMMA?
    BEQ INPUT5
    CMP A #$D IS IT A C.R.?
    BNE INPUT4
    STA A CRFLAG SET FLAG
    JSR PCRLF OUTPUT A CR & LF
INPUT5 LDX #BUFFER SET POINTER
    JSR BCDCON GO CNVRT NUM.
    LDX XTEMP4
    BSR LABLS2
    STX BUFPNT SAVE POINTER
INPUT6 CMP A #' , IS IT A COMMA?
    BNE INPUT7
    INX
    LDA A CRFLAG TEST FLAG
    BEQ INPUT1
    BRA INPUT0
INPUT7 JSR TSTTRM
    BNE INPUT9
INPUT72 LDA A CRFLAG TEST FLAG
    BEQ INPUT8
INPUT75 JMP RUNEXC
INPUT8 JSR INCH GET CHAR.
    CMP A #$D C.R.?
    BNE INPUT8
    JSR PCRLF
    BRA INPU75
INPUT9 LDA A #$45
    JMP MISTAK REPORT ERROR

```

* GET AND PUT LABLE

```

LABELS JSR FNDLBL GO FIND IT
LABLS2 JSR PUTLBL GO PUT IT
    JMP NXTSPC GET TO NEXT SET

```

* DATA ROUTINE

```

DATA LDA A RUNFLG RUNNING?
    BEQ READ6
    JSR NXTSPC FIND NEXT
    STA A DATAFL SET DATA FLAG
    STX DATAST SET POINTER
    STX DATAPT
    BRA READ6 RETURN

```

* READ DATA ROUTINE

```
READ LDA A RUNFLG RUNNING?
    BEQ READ6
    LDA A DATAFL CHECK FLAG
    BEQ READ8
    JSR NXTBLK GET NEXT
READ2 JSR SKIPSP GO CLASSIFY
    JSR FNDLBL
    STX XTEMP4
    LDX BUFPNT
    STX XTEMP5 SAVE IT
    LDX DATAPT GET DATA PNTR
    JSR EXPR GET DATA
    LDA A 0,X GET CHAR.
    JSR TSTTRM TEST IT
    BNE READ25
    LDX DATAST SET POINTER
    BRA READ3
READ25 INX BUMP THE POINTER
READ3 STX DATAPT
    LDX XTEMP5
    STX BUFPNT
    LDX XTEMP4
    BSR LABLS2
    CMP A #', IS IT A COMMA?
    BNE READ4
    INX
    BRA READ2 REPEAT
READ4 JSR TSTTRM
    BNE READ8 ERROR
READ6 JMP RUNEXC RETURN
READ8 LDA A #$51
    JMP MISTAK
```

* RESTORE DATA STRING

```
RESTOR STX XSAVE SAVE POINTER
    LDX DATAST
    STX DATAPT FIX DATA PNTR
    LDX XSAVE RESTORE POINTER
    BRA READ6
```

* ON GOTO ROUTINE

```
ONGOTO JSR NXTBLK FIND NEXT BLOCK
    JSR EXPR EVAL. EXPR.
    LDA A NUMBER+2
    AND A #$0F MASK L.S. DIGIT
```



```

PSH A SAVE A
CLR CRFLAG
INX BUMP THE POINTER
INX
LDA A 0,X GET CHAR
CMP A #'T IS IT A "T"?
BEQ ONGOTO
STA A CRFLAG SET FLAG
ONGOTO JSR NXTBL4 GET NEXT
STX XSAVE SAVE X
PUL A RESTORE A
ONGOT1 DEC A
BEQ ONGOT4
ONGOT2 LDA B 0,X GET A CHAR,
INX BUMP THE POINTER
CMP B #', IS IT A COMMA?
BNE ONGOT3
STX XSAVE SAVE THE POINTER
BRA ONGOT1 REPEAT
ONGOT3 CMP B #$D C^R^ ?
BNE ONGOT2
LDX XSAVE RESTORE POINTER
ONGOT4 LDA B CRFLAG CHECK FLAG
BEQ ONGOT6
JMP GOSUB2
ONGOT6 JMP GOT01

```

* ROUTINE

```

IF JSR NXTSPC FIND NEXT
JSR EXPR EUAL EXPR
LDA A 0,X GET CHAR
BSR CLSREL REL OPERATOR?
BNE IF9 ERROR!
PSH A SAVE A
LDA A 1,X GET CHAR
BSR CLSREL REL OP?
PUL A RESTORE A
BNE IF1
LDA B 1,X
ABA FORM REL CODE
INX BUMP THE POINTER
IF1 INX
PSH A SAVE A
JSR EXPR EVAL EXPR
PUL A
AND A #$0F MASK
SUB A #9 BIAS IT
BMI IF9 ERROR?

```

```

ASL A TIMES FOUR
ASL A
STA A OFSET3+1
JSR SUB GO COMPARE
JSR ZCHK SET CC REG
OFSET3 BRA *
BRATBL BLE IF4 BRANCH TABLE
BRA IF8
BNE IF4
BRA IF8
BGE IF4
BRA IF8
BLT IF4
BRA IF8
BEQ IF4
BRA IF8
BGT IF4
BRA IF8
BRA IF9 ERROR!
IF4 LDX BUFPNT SET POINTER
LDA A 0,X GET CHAR
CMP A #'T IS IT A "T"?
BNE IF6
JSR NXTSPC
STX BUFPNT SAVE POINTER
JSR CLASS1 GO CLASSIFY
CMP B #3 IS IT A NUMBER?
BNE IF6
JMP GOTO1 GO TO GOTO
IF6 JMP RUNEX3
IF8 JMP RUNEXC GO PROCESS CMND
IF9 LDA A #$62 SET ERROR
JMP MISTAK

```

* CLASSIFY RELATIONAL OPERATION

```

CLSREL CMP A #$3B
BLS CLSRE5
CMP A #$3E CHECK CHAR
BHI CLSRE5
CLR B CLEAR FLAG
RTS RETURN
CLSRE5 INC B SET FLAG
RTS RETURN

```

* GOSUB ROUTINE

```

GOSUB LDA B RUNFLG
BEQ IF8

```

```

JSR NXTSPC FIND NEXT
GOSUB2 INC SUBCNT
JSR EXPR EVALUATE EXPR
DEX
JSR FNDCRT FIND C.R.
INX BUMP THE POINTER
LDA A 0,X GET LINE NO
PSH A
LDA A 1,X
PSH A SAVE AS RET. ADD.
STS CPX1 SAVE SP
LDX #STKBOT+35
JSR CMPX CHECK BOUNDS
BLS GOSUB4
JMP ADJEN2 RPT OVFL
GOSUB4 JMP GOTO2

```

* RETURN ROUTINE

```

RETURN LDA A #$73
DEC SUBCNT DEC COUNTER
BPL RETUR2
JMP MISTAK ERROR!
RETUR2 PUL A GET RET. ADD.
PUL B
JSR FINDLN GO FIND LINE
JMP GOTO3

```

* EXPRESSION EQUATE

```

EXPEQU JSR FNDLB0 FIND LABLE
STX XTEMP4 SAVE
JSR NXTSPC
INX
JSR EXPR GO EVALUATE
LDX XTEMP4 GET POINTER
JMP PUTLBL INSTALL

```

* FOR ROUTINE

```

FOR JSR NXTBLK FIND NEXT
PSH A
BSR EXPEQU
LDX DIMPNT
STX CPX1
LDX FORSTK
PUL A
STA A 0,X
LDA A BUFPNT+1

```

```

DEX DEC THE POINTER
STA A 0,X
LDA A BUFPNT SET UP INDEX
DEX
STA A 0,X
DEX
JSR CMPX CHECK FOR OVFLW
BHI FOR5
JMP ADJEN2
FOR5 STX FORSTK SAVE POINTER
JMP RUNEXC

```

* NEXT ROUTINE

```

NEXT JSR NXTBLK FIND NEXT
STX NXPNTR
LDX FORSTK SET POINTER
NEXT1 CPX MEMEND OVFLW?
BNE NEXT2
LDX BUFPNT RESTORE PNTR
BRA NEXT9 ERROR!
NEXT2 INX FIXUP POINTER
INX
INX
CMP A 0,X CHECK
BNE NEXT1
DEX FIX POINTER
DEX
DEX
STX FORSTK
INX
LDX 0,X
STX BUFPNT SAVE IT
JSR FNDLBL FIND LABLE
STX XTEMP4 SAVE IT
JSR NXTSPC FIND NEXT
JSR EXPR EVALUATE
JSR STAKUP
LDX XTEMP4 RESTORE PNTR
JSR GETVAL GET LABLE VALUE
LDX BUFPNT
LDA A 0,X GET CHAR
CMP A #'S IS IT STEP?
BEQ NEXT4
JSR UPSCLR
INC A
STA A NUMBER+2
BRA NEXT5
NEXT4 JSR NXTSP4

```

```

JSR EXPR
LDA A NUMBER
STA A LETFLG SHOW NEG.
NEXT5 JSR ADD GO ADD IN STEP
LDX #TRYVAL SET POINTER
JSR PUTLBL SAVE LABLE
JSR SUB COMPARE
JSR ZCHK SET CC REG
LDA B LETFLG CHK FLAG
BMI NEXT6
TAP SET CC
BGE NEXT8
BRA NEXT7
NEXT6 TAP SET CC
BLE NEXT8
NEXT7 LDX FORSTK
INX FIXUP PNTR
INX
INX
STX FORSTK SAVE IT
LDX NXPNTR
STX BUFPNT SAVE
BRA NEXT85
NEXT8 LDX #TRYVAL
JSR GETVAL
LDX XTEMP4
JSR PUTLBL
NEXT85 JMP RUNEXC
NEXT9 LDA A #$81 SET ERROR
NEXT10 JMP MISTAK

```

* EXPRESSION HANDLER

```

EXPR CLR STKCNT SET COUNT = 0
EXPRO LDA A STKCNT
STA A AUXCNT
BSR EVAL
TST A CHECK FOR ERROR
BNE NEXT10
EXPR1 RTS RETURN
*
**EVAL
* EVALUATE AN ALGEBRAIC STRING
*
EVAL STS STKTOP SAVE SP TOP
EVA0A JSR SKYCLS
STX BUFPNT
CMP B #1 SEE IF EMPTY EXPRESSION
BNE EVAL0

```

```

LDA A #$21
BRA EVAL3
EVAL0 LSR B SET UP
CMP B #3 CHECK FOR UNARY + OR -
BNE EVAL1
JSR UPSCLR
EVAL1 LDX BUFPNT
EVAL1A JSR SKYCLS GET NEXT CHAR
STX BUFPNT
CMP B #4 CHECK FOR OPERATORS
BLS EVAL1Z
LDA B #5 SET UP
EVAL1Z ASL B
STA B OFFREL+1 SET UP BRANCH
OFFREL BRA *
BRA EVAL2 ERROR
BRA EVAL4 TERMINATOR
BRA EVAL8 LETTER
BRA EVAL7 NUMBER
BRA EVAL1C RIGHT PAREN
PSH A SAVE
INX
BRA EVA0A AGAIN
EVAL1C TSX GET SP
DEX ADJUST
LDA B DIMFLG
CPX STKTOP CHECK FOR EMPTY
BEQ EVAL1E
PUL A
CLR B
CMP A #'( CHECK FOR L PAREN ON STACK
BEQ EVAL1C IF SO, OK
EVAL1E TST B CHECK FOR ALRIGHT
BEQ EVAL2 IF NOT SET, ERROR
EVAL4 CLR A
LDA B STKCNT GET STACK STKCNT
DEC B CHECK OP STACK
CMP B AUXCNT
BNE EVAL2 IF NOT EMPTY, ERROR
TSX
DEX ALIGN
CPX STKTOP CHECK OPERATOR STACK
BEQ EVAL3A IF NOT EMPTY° ERROR
EVAL2 LDA A #$20 SET ERROR NUMBER
EVAL3 LDS STKTOP GET SP
EVAL3A LDX BUFPNT SET POINTER
RTS
EVAL7 JSR STAKUP SHIFT OP STACK UP
LDX BUFPNT

```

```

JSR BCDCON GET OPERAND
BRA EVAL12
EVAL8 LDA A 1,X GET NEXT CHAR
JSR CLASS1 GO CLASSIFY
CMP B #2 CHECK FOR LETTER
BNE EVAL9 IF NOT, VARIABLE
LDA A 0,X GET CHAR BACK
STX XSAVE SET FOR ENTRY TO FIMDKY
LDX #FCTTBL
JSR FNDKE2 GO CHECK FUNCTION
TST A CHECK SUCCESS
BEQ EVAL4
JMP RUNEX4 GO SERVICE
EVAL86 LDA A #'? GET STGNUM OPERATOR
EVAL87 PSH A PUT ON STACK
LDX XSAVE
JMP EVA0A
EVAL85 LDA A #'@ GET ABS OPERATOR
BRA EVAL87
EVAL88 JSR UPSCLR MOVE STACK UP
JSR RANDOM COMPUTE RANDOM #
STA A NUMBER+2
EVAL89 LDX XSAVE RESTORE POINTER
BRA EVAL12
EVAL9 LDA B STKTOP
PSH B
LDA B STKTOP+1
PSH B
LDA B AUXCNT GET COUNTER
PSH B SAVE
LDA B DIMFLG GET FLAG
PSH B SAVE
JSR FNDLBO FIND VARIABLE STORAGE
PUL B GET FLAG
STA B DIMFLG RESTORE
PUL B GET COUNTER
STA B AUXCNT RESTORE
PUL B
STA B STKTOP+1
PUL B
STA B STKTOP
JSR STAKUP
LDX XTEMP
JSR GETVAL MOVE VALUE TO NUMBER
BRA EVA12A
EVA11C LDX BUFPNT RESTORE POINTER
INX
EVAL12 STX BUFPNT SAVE POINTER
EVA12A TSX

```

```

DEX
CPX STKTOP CHECK OPERATOR STACK
BEQ EVAL10 IF EMPTY, DON'T OPERATE
PUL A
PSH A PUT BACK
CMP A #'( CHECK FOR LEFT PAREM
BEQ EVAL10 IF SO, DON'T OPERATE
JSR CLASS1 GO CLASSIFY
PSH B
LSR B SET UP ID
LDA A STKCNT GET COUNT
DEC A
CMP B #4 CHECK FOR ABS OR SON
BEQ EVA12C IF SO, GO AHEAD
CMP A AUXCNT OTHERWISE CHECK FOR 2 OPERANDS
BEQ EVAL10 IF NOT, ABORT
EVA12C CMP A #9 CHECK OVERFLOW
BLS EVA12D OK
LDA A #$24 SET ERROR
BRA EVAL19
EVA12D PUL A GET CLASSIFICATION
PUL B GET OPERATOR
SUB A #6 REMOVE BIAS
ASL A #2
STA A OPOFF+1 SET UP JMP
LDX #OPTBL POINT
OPOFF LDX 0,X
JSR 0,X GO OPERATE
JSR ZCHK CHECK RESULT
BVC EVA12A IF NO OVFL, GO OPERATE AGAIN
EVAL18 LDA A #$23 SET ERROR NUMBER
EVAL19 JMP EVAL3
EVAL10 JMP EVAL1
OPTBL FDB ADD
FDB SUB
FDB SIGNUM
FDB ABSVAL
FDB MULT
FDB DIVIDE
FDB EXPON
*
** GET VALUE
* MOVE 3 BYTES POINTED TO BY X TO NUMBER
*
GETVAL LDA A 0,X GET VALUE
STA A NUMBER STORE
LDA A 1,X
STA A NUMBER+1
LDA A 2,X

```



```

    STA A NUMBER+2
    RTS
*
*
** STACKUP
* ROLL OPERATIONAL STACK UPWARD
*
STAKUP LDX #STKEND POINT TO END
STAKU2 LDA B 3,X
    STA B 0,X MOVE
    INX
    CPX #NUMBER SEE IF DONE
    BNE STAKU2
    INC STKCNT
    RTS
*
*
** STACKDOWN
* ROLL OPERATIONAL STACK DOWNWARD
*
STAKDN LDX #AX-1 POINT TO STORE
STAKD1 LDA B 0,X
    STA B 3,X
    DEX
    CPX #STKEND-1 SEE IF DONE
    BNE STAKD1
    DEC STKCNT
    RTS
*
*
** UADD
* UNSIGNED ADD OF AX TO NUMBER
*
UADD CLC ZERO THE CARRY
UADD1 LDX #NUMBER+2 POINT TO STORE
UADD2 LDA A 0,X GET ADDEND
    ADC A 3,X ADD IN AUGEND
    DAA
    STA A 0,X SAVE
    DEX
    CPX #NUMBER-1 SEE IF DONE
    BNE UADD2
UADD22 PSH B
    LDA B #$02 SET FOR OVFL
    BIT A #$F0 AND AGAIN
    BNE UADD25
    CLR B RESET OFVL
UADD25 ORA B OVFLBF
    STA B OVFLBF SET OVFL IF NECESSARY

```

```

TBA
PUL B
UADD3 RTS
*
*
**USUB
* UNSIGNED SUBTRACT OF AX FROM NUMBER
*
USUB BSR TENCOM GO TEN'S COMPLEMENT
SEC FIX UP
BRA UADD1 GO ADD
*
*
**TENCOM
* UNSIGNED TEN'S COMPLEMENT OF AX (ALMOST)
*
TENCOM LDX #AX+2 POINT TO AX
TENC01 LDA A #$99
SUB A 0,X SUBTRACT FROM 99
STA A 0,X SAVE
DEX
CPX #AX-1
BNE TENC01
AND A #$0F RESET SIGN
STA A 1,X STORE
RTS
*
*
** SET SIN
* CALCULATE RESULT SIGN
*
SETSIN CLR OVFLBF CLEAR OVFL INDICATOR
SETSIO LDA A AX GET SIGN
TAB SAVE
AND B #$0F RESET SIGN
STA B AX PUT BACK
STA A AXSIGN SAVE SIGN
EOR A NUMBER FORM NEW SIGN
STA A SIGN SAVE
ABSVAL LDA B NUMBER GET MS BYTE
AND B #$0F RESET SIGN
STA B NUMBER PUT BACK
TST A TEST NEW SIGN
RTS
*
*
**
* SUBTRACT AX FROM NUMBER
*

```

```

SUB LDA A NUMBER GET MS BYTE
  EOR A #$F0 CHANGE SIGN
  STA A NUMBER PUT BACK
* GO INTO ADD
*
*
* ADD
* ADD AX TO NUMBER
*
ADD BSR RELAY
  BSR SETSIN GO CALCULATE SIGN
  BPL ADD0 USE EITHER SIGN
  BSR USUB OTHERWISE SUBTRACT
  TAP SET CCR
  BVC ADD1 CHECK OVERFLOW
  COM AXSIGN CHANGE FOR AX SMALLER
  BRA ADD15
ADD0 BSR UADD GO ADD
  BRA ADD2 GO FIX SIGN
ADD1 BSR RELAY COPY NUMBER TO AX
  JSR UPSCLR RESTORE
  BSR USUB GO NEGATE
ADD15 CLR OVFLBF
ADD2 LDA A AXSIGN GET OLD SIGN
*
*
** FIXSIN
* SET THE SIGN ON THE RESULT
*
FIXSIN AND A #$F0 MASK
  LDA B #$0F SET MASK
  AND B NUMBER RESET SIGN
  ABA TACK ON SIGN
  STA A NUMBER PUT BACK
FIX2 RTS
*
*
** MULT
* MULTIPLY AC BY AX
*
MULT BSR RELAY MOVE STACK
  BSR SETSIN GO CALC. SIGNS
MULT0 JSR UPSCLR MOVE STACK UP
  LDA B #5 SET COUNTER
MULT1 LDA A AC GET MS BYTE OF AC
  BEQ MULT3 IF ZERO , LOOP
MULT2 JSR UADD ADD IN AX
  DEC AC ONCE DONE
  BNE MULT2

```

```

MULT3 DEC B ONCE DONE
    BEQ MULT4 CHECK IF ALL DONE
    BSR ACLEFT SHIFT AC LEFT
    LDA A NUMBER
    JSR UADD22
    BRA MULT1
*
*
** DIVIDE
* DIVIDE AC-NUMBER BY AX
*
DIVIDE BSR RELAY
    LDX #AX
    JSR ZCHK1 GO CHECK IF AX=0
    BNE DIVID1 IF NOT, OK
DIVID0 LDA A #$22 SET ERROR
    JMP EVAL3
RELAY JMP STAKDN RELAY TO STACK DOWN
DIVID1 JSR SETSIN CALC, SIGNS
    JSR STAKUP PUSH BACK
    BSR ACLEFT SHIFT DOWN
    CLR 2,X
    CLR 3,X ZERO OUT NUMBER
    LDA B #5 SET LOOP COUNT
DIVID2 BSR ACLEFT MOVE AC DOWN
DIVI2A JSR TENCOM TAKE 10'S COMP
DIVID3 BSR DADD GO SPECIAL ADD
    BIT A #$F0 CHECK FOR OVERFLOW
    BNE DIVID4
    JSR TENCOM IF SO, RESTORE AX
    CLC
    BSR DADD1 ADD BACK IN
    DEC B ONE PASS MADE
    BNE DIVID2
MULT4 LDA A SIGN GET THE SIGN
    BSR FIXSIN GO FIX UP THE SIGN
    LDX #AC-1 POINT TO AC
    JMP STAKD1 MOVE STACK BACK
DIVID4 INC NUMBER+2 ADD ONE IN
    BRA DIVID3 GO DO AGAIN
*
*
** ACLEFT
* SHIFT AC-NUMBER LEFT 4 BITS
*
ACLEFT LDA A #4 SET FOR 4 BITS
ACLEF1 LDX #AX-1 POINT X
    CLC
ACLEF2 ROL 0,X ROTATE

```

```

DEX
CPX #AC-1 CHECK IF DONE
BNE ACLEF2
DEC A CHECK FOR DONE
BNE ACLEF1
RTS
*
*
** DADD
* ADD AX TO A C
*
DADD SEC
DADD1 LDX #AC+2
  LDA A AC GET MS BYTE
  AND A #$0F RESET SIGN
  STA A AC STORE BACK
DADD2 LDA A 0,X GET ADDEND
  ADC A 6,X ADD IN
  DAA
  STA A 0,X SAVE
  DEX
  CPX #AC-1 SEE IF DONE
  BNE DADD2
  RTS
*
** SIGNUM
* CALCULATE SIGNUM FUNCTION
*
SIGNUM BSR ZCHK GO CHECK = 0
  BEQ SIGNU2 IF SOY RESULT =0
  LDA B NUMBER OTHERWISE GET SIGN
SIGNU1 BSR SIGNU2 GO CLEAR
  INC NUMBER+2 MAKE = I
  TBA SET FOR FIXSIN
  JMP FIXSIN GO SET THE SIGN
SIGNU2 JMP CLRNUM
*
*
** EXPON
* CALCULATE EXPONENTIATION
* ONLY POSITIVE EXPONENTS UP TO 99 ALLOWED
*
EXPON BSR RELAY MOVE OPERANDS DOWN
  CLR B
  STA B OVFLBF CLEAR OVER FLOW
  LDA A AX+2 GET EXPONENT
  BEQ SIGNU1 IF 0, GO MAKE RESULT +1
  JSR STAKUP GET TWO COPIES
  BSR RELAY MOVE DOWN

```

```

EXPON1 ADD A #$99 DECREMENT
DAA
BEQ CMPX2 WHEN 0 ALL DONE
PSH A SAVE EXP
JSR SETSIO GO FIX SIGNS
JSR MULT0 GO MULTIPLY
PUL A GET EXPONENT
BRA EXPON1 LOOP
*
*
** CMPX
* FULL COMPARE ON X
* COMPARES X WITH CONTENTS OF CPX1
*
CMPX STX CPX2 SAVE
CMPX1 LDA A CPX2 GET MS BYTE
CMP A CPX1 COMPARE
BNE CMPX2 IF NOT EQUAL, DONE
LDA B CPX2+1 GET LS BYTE
CMP B CPX1+1 COMPARE
CMPX2 RTS DOME
*
*
** ZCHK
* CHECK OPERAND FOR EQUAL TO 0
*
ZCHK LDX #NUMBER
ZCHK1 CLR B
TST 2,X
BNE ZCHK2
TST 1,X
BNE ZCHK2
LDA A 0,X GET MS BYTE
AND A #$0F
BNE ZCHK2 CHECK FOR 0
STA A 0,X RESET SIGN BITS
LDA B #4
ZCHK2 LDA A 0,X GET MS BYTE
ROR A MOVE A SIGN BIT TO N
AND A #8 MASK N BIT
ABA MERGE Z AND N
ORA A OVFLBF ADD IN V
TAP SET CCR
RTS
*
*
**
SKYCLS JSR SKIPSP
BRA CLASS1

```

```

*
*
**CLASS
*CLASSIFY A CHARACTER IN THE A ACCUMULATOR
*CLASSIFICATION RETURNED IN B
* 0 ERROR
* 1 TERMINATOR
* 2 LETTER
* 3 NUMBER
* 4 )
* 5 (
* 6 +
* 7 -
* 8 SGN
* 9 ABS
* 10 *
* 11 /
* 12 ~

```

```

CLASS LDA A 0,X GET CHAR
CLASS1 LDA B #1 SET UP
    CMP A #$D CHECK FOR CR
    BEQ CLAS25
    DEC B
    PSH A SAVE CHAR
CLAS2B SUB A #'( REMOVE BIAS
    BMI CLASS2 CHECK ILLEGAL
    CMP A #'@- '( CHECK LIMIT
    BLS CLASS3 NOT LETTER
    CMP A #'Z- '( CHECK FOR LETTER
    BLS CLAS1B
    CMP A #'^- '( CHECK FOR ILLEGAL
    BNE CLASS2
    LDA B #10 FIX UP
CLAS1B ADD B #02
CLASS2 PUL A RESTORE CHARACTER
CLAS25 RTS DONE
CLASS3 STX XSAVE2 SAVE X REG
    LDX #CLSTBL POINT TO TABLE
    STA A CLSOFF+1 SET BIAS
CLSOFF LDA B 0,X GET CLASSIFICATION
    LDX XSAVE2 RESTORE X REG,
    BRA CLASS2
CLSTBL FCB 5,4,10,6,1,7,0,11,3,3,3,3
    FCB 3,3,3,3,3,3,1,1,1,1,1,8,9

```

```

*
*
* RANDOM GENERATOR
*
RANDOM LDA B #8 SET COUNTER

```

```
LDX #RNDM
RPT LDA A 3,X GET M.S. BYTE OF RANDOM NO.
ASL A SHIFT IT LEFT THREE:
ASL A TIMES TO GET BIT 28
ASL A IN LINE WITH BIT 31
EOR A 3,X XOR A WITH RANDOM NO
ASL A PUT BIT 28.XOR31 IN
ASL A CARRY BY SHIFTING LEFT
ROL 0,X ROTATE ALL FOUR BYTES OF
ROL 1,X THE RANDOM NO, ROTATING
ROL 2,X THE CARRY INTO THE LSB
ROL 3,X THE MSB IS LOST
DEC B DECREMENT THE COUNTER
BNE RPT IF ITS NOT 0, GO REPEAT
LDA A 0,X PUT RANDOM # IN A
CMP A #$9F CHECK IN RANGE
BHI RANDOM IN NOT GET ANOTHER
ADD A #0 SET HALF CARRY
DAA
RTS
ENDSTR RMB 2
STORSP EQU *
```

```
ORG EXTERN
RTS
END
```